

短絡評価 — && と || の本当の意味

今日のゴール

&& と || を、C の規格どおり必要なときだけ右辺を評価する形に直す。
`if (p != 0 && p->val > 0)` のような、いまは書けない書き方が書けるようになる。

1 この回の位置づけ

- 意味論発展シリーズの第 1 回（選択制）。前提はコマ 8 まで
- いまの実装が C とずれている箇所を、自分で見つけて直す回
- `mycc.py` は書き換えない。ラッパー `semcc.py`（完成品）が `And / Or` の生成だけ差し替える
- 編集するのは `shortcircuit.py` の 2 つの関数

2 いまのコンパイラは何をしているか

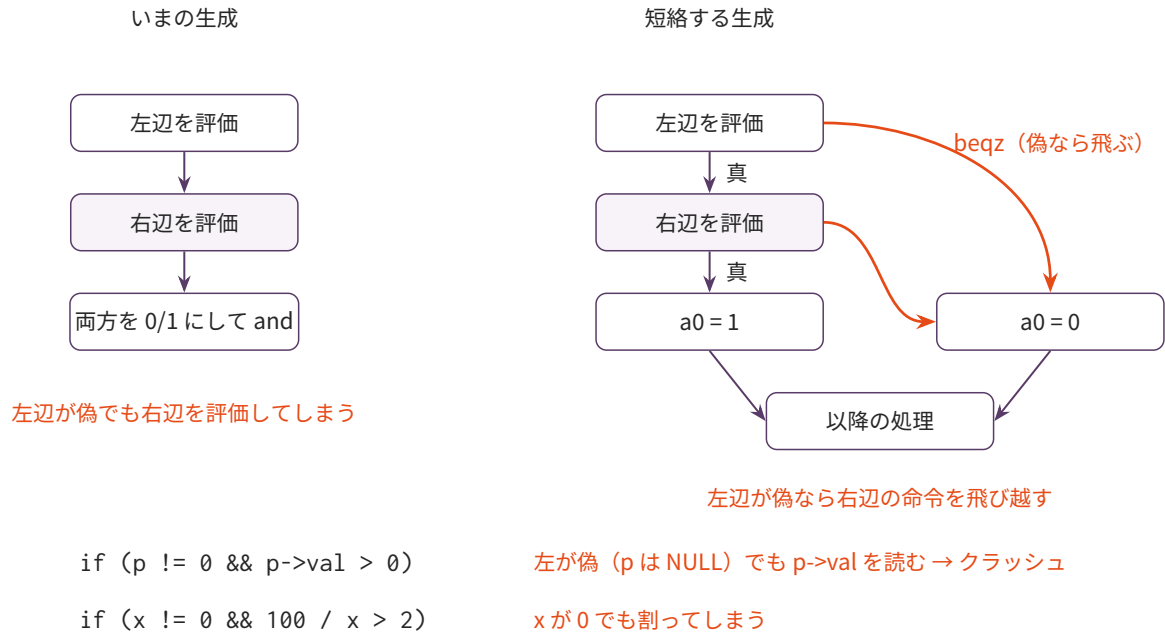
`a && b` に対して、いまのコード生成はこうなっている。

```
# 左辺を評価して a0 へ
# a0 をスタックへ退避
# 右辺を評価して a0 へ
# 退避した左辺を a1 へ
snez a1, a1      # 左辺を 0/1 に
snez a0, a0      # 右辺を 0/1 に
and a0, a1, a0
```

両辺を必ず評価している。結果の値（1 か 0）は正しいので、コマ 5 以来のテストはすべて通る。だから誰も気づかない。

しかし C の規格では、&& と || は特別扱いされている。

左辺だけで結果が決まる場合、右辺は評価されない（short-circuit evaluation）。



短絡は「速くするための工夫」ではなく、Cの言語仕様そのもの。この形の書き方が成立するかどうかが変わる

図1 いまの生成と、短絡する生成

これは「速くするための最適化」ではなく、言語の意味そのものである。右辺が評価されないことに依存した書き方が、Cでは日常的に使われるからである。

3 何が壊れるのか

いまの実装で次を書くと、実際に落ちる。

```

if (p != 0 && p->val > 0) { ... } /* p が NULL でも p->val を読む → クラッシュ */
if (i < n && a[i] == x) { ... } /* 範囲外アクセス */
if (x != 0 && 100 / x > 2) { ... } /* x が 0 でも割る */
    
```

1行目は、コマ13の連結リストで自然に書きたくなる形である（実際、コマ13のlist_sumはwhile (head != 0)とhead->valを2つの文に分けて書いてある。分けざるを得なかった、とも言える）。

|| も同じで、if (p == 0 || p->val == 0)は「NULLなら真、そうでなければ中身を見る」という定番の書き方である。

重要

このずれは、テストが通ってしまうのが厄介なところである。値としては正しいので、コマ5~16のどのテストでも露見しない。p != 0 && p->val という形を書いた瞬間に、初めて牙をむく。

「テストが通る = 正しい」ではない。テストは書いたことしか確かめない。

4 どう直すか 一分岐で作る

短絡評価は、算術ではなく制御フローである。コマ5のifと同じく、ラベルとbeqzで作る。

```
a && b:
    左辺を評価          → a0
    beqz a0, Lfalse     ← 左が偽なら、右辺の命令を飛び越す
    右辺を評価          → a0
    beqz a0, Lfalse
    li a0, 1
    j Lend
Lfalse:
    li a0, 0
Lend:
```

||はこの裏返しで、bnez (0以外なら飛ぶ)を使い、左辺が真ならLtrueへ飛んで1にする。

結果が1か0に正規化される点は、いままでと同じである(だからfixed15は通り続ける)。

補足

なぜ&&は「演算子」なのに分岐が要るのか。+や*は、両辺の値が揃ってはじめて計算できる。一方&&は、左辺の値によって右辺を評価するかどうかが変わる。これはifと同じ性質で、実は演算子の顔をした制御構文である。同じ仲間に三項演算子a ? b : cがあり、こちらも「条件によって片方しか評価しない」ため、分岐で生成する。

5 実装

Step	実装対象	内容
1	gen_and	beqz で右辺を飛ばす
2	gen_or	bnez で右辺を飛ばす

`cg.new_label()` (コマ 5 で使ったラベル生成) で飛び先を 2 つ作る。

```
python3 semcc.py tests/null_guard.c # 生成結果を見る
python3 semcc.py tests/null_guard.c | grep beqz
```

6 テスト

```
python3 check.py # 生成された命令列の構造を確認 (未実装は SKIP)
python3 golden.py # 短絡が要るテスト + fixed15
```

`golden.py` は 3 つを続けて確認する。

1. 短絡なし (いまの `mycc`) では `tests/` が落ちること
2. 短絡ありで全部通ること
3. `fixed15` が壊れていないこと

`tests/` の中身は次の 4 本である。

テスト	確認内容
<code>null_guard.c</code>	<code>p != 0 && p->val</code> が NULL でも落ちない
<code>div_guard.c</code>	<code>x != 0 && 100 / x</code> がゼロ除算しない
<code>value.c</code>	<code>&& / </code> が返す値 (1 か 0) が正しい
<code>side_effect.c</code>	右辺の関数呼び出しが実際に呼ばれない (グローバル変数で観測)

`side_effect.c` が本質的である。「評価されないこと」は、副作用を観測してはじめて確かめられる。

7 発展課題

1. **！との組み合わせ**: `!(a && b)` と `!a || !b` (ド・モルガン) が同じ結果になることを確かめる。ただし評価される回数は違う。どう違うか
2. **三項演算子**: `a ? b : c` を追加する (Core 外だが、生成の形は `&&` とほぼ同じ)
3. **B1 の定数畳み込みと組み合わせ**: `0 && f()` は畳み込んでよいか。 `f()` に副作用があっても消してよいのか考える (C の規格では、短絡により `f()` は呼ばれないので消してよい)
4. **命令数を測る**: `count_insns.py` で短絡あり／なしを比べる。条件が複雑なプログラムでは、短絡版のほうが命令数は増えることもある (分岐とラベルが増えるため)。それでも短絡が正しい理由を説明する

補足

コラム: 言語ごとの違い。 `&&` / `||` の短絡は C・Java・Python などほとんどの言語にあるが、短絡しない論理演算子を別に持つ言語もある (Ada の `and` / `and then`、VB の `And` / `AndAlso`、Pascal の処理系オプションなど)。「両方必ず評価してほしい」場面 (両辺に副作用がある場合) があるからである。C では、短絡しない版がほしければビット演算 `&` / `|` を使う — ただし値が 0/1 に正規化されないので、意味は微妙に違う。